

# Docker and Containerization

Created by : [Manoj Bhatta](#) | [LinkedIn](#)

*A complete, beginner-friendly deep dive into containers, Docker, and production deployment from zero to CI/CD.*

## 1. What is Containerization?

Imagine you've built an application on your laptop. It works perfectly. You hand it to a colleague, and it **breaks**. Different OS, different library versions, different configurations. This is the classic **"it works on my machine"** problem.

**Containerization solves this.** It packages your application *along with everything it needs* code, runtime, libraries, system tools, and settings into a single, portable unit called a **container**.

### How is it different from a Virtual Machine?

| Feature           | Virtual Machine (VM)               | Container                                       |
|-------------------|------------------------------------|-------------------------------------------------|
| Abstraction level | Virtualizes the entire hardware    | Virtualizes only the OS layer                   |
| Includes          | Full guest OS + app + dependencies | App + dependencies only (shares host OS kernel) |
| Size              | Gigabytes (heavy)                  | Megabytes (lightweight)                         |
| Startup time      | Minutes                            | Seconds                                         |
| Isolation         | Strong (separate kernel)           | Process-level (shared kernel)                   |
| Use case          | Running different OSes on one host | Running multiple isolated apps efficiently      |

### The Core Idea

Containers use two Linux kernel features to achieve isolation:

1. **Namespaces** — Isolate *what a process can see*. Each container gets its own view of the system: its own process tree, network stack, mount points, and

user IDs.

2. **cgroups (Control Groups)** — Limit *what a process can use*. CPU, memory, disk I/O, and network bandwidth can be capped per container.



**Think of it this way:** A VM is like renting an entire apartment (full OS). A container is like renting a desk in a co-working space — you share the building's infrastructure but your workspace is isolated.

## 2. What is Docker?

**Docker** is the most popular platform for building, shipping, and running containers. It didn't *invent* containerization (Linux had LXC containers before Docker), but it made containers **accessible, standardized, and developer-friendly**.

### Docker: The Brand vs The Technology

This is a common point of confusion. Let's break it down clearly:

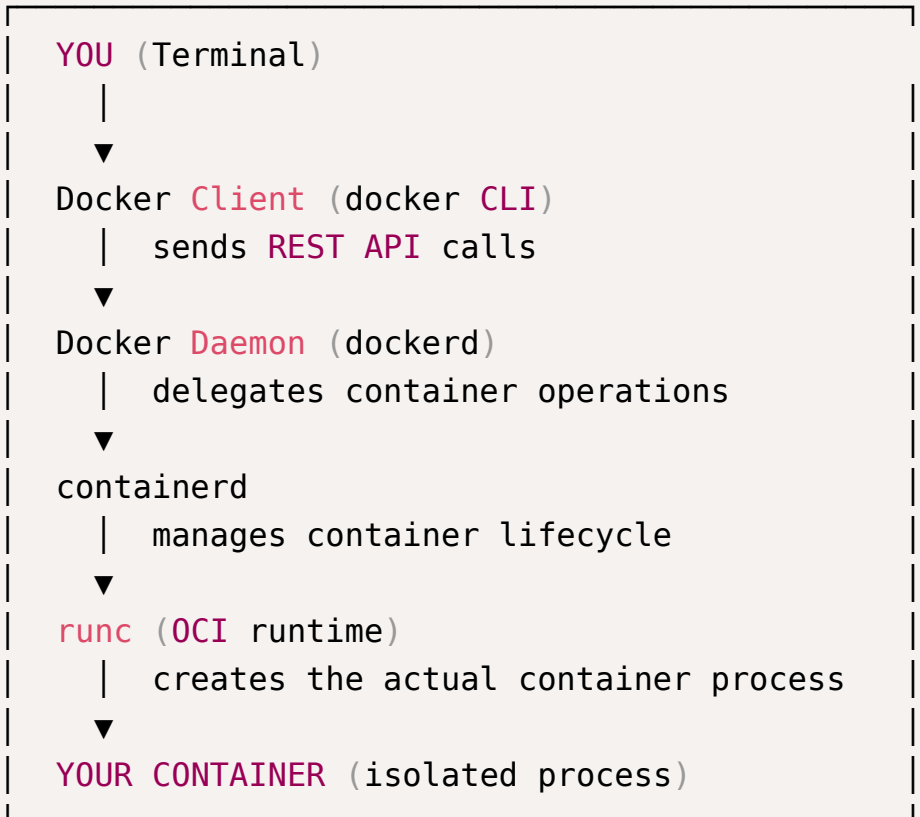
| Term                  | What it is                                                                                                                                     |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Docker, Inc.</b>   | The <i>company</i> behind Docker. They build commercial products like Docker Desktop and Docker Hub.                                           |
| <b>Docker Engine</b>  | The <i>open-source runtime</i> that actually builds and runs containers. This is the core technology.                                          |
| <b>Docker Desktop</b> | A <i>GUI application</i> (for Mac, Windows, Linux) that bundles Docker Engine + CLI + Compose + Kubernetes + a VM layer for non-Linux systems. |
| <b>Docker Hub</b>     | A <i>cloud registry</i> where you store and share container images. Think of it as "GitHub for container images."                              |
| <b>Docker CLI</b>     | The <i>command-line tool</i> ( <code>docker</code> ) you type commands into.                                                                   |

### Docker Engine Architecture

Docker Engine has three major components working together:

1. **Docker Daemon ( `dockerd` )** — The background service that does the heavy lifting. It manages images, containers, networks, and volumes. It listens for API requests.

2. **Docker Client** ( `docker` ) — The CLI you interact with. When you type `docker run` , the client sends that request to the daemon via a REST API.
3. **containerd** — A lower-level container runtime that Docker delegates to. It manages the complete container lifecycle: image transfer, container execution, storage, and networking.



**Key takeaway:** When you type `docker run` , you're talking to the **client**, which talks to the **daemon**, which uses **containerd**, which uses **runc** to create the actual isolated process. Each layer has a specific responsibility.

## 3. Where Can You Run Docker?

Docker runs on multiple platforms, but the experience differs depending on your OS because containers are a **Linux-native technology**.

### 3.1 Linux (Native)

This is Docker's home turf. Containers share the host Linux kernel directly no virtualization overhead.

```
# Install Docker Engine on Ubuntu
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io

# Verify installation
docker --version
docker run hello-world
```

**Pros:** Best performance, no VM overhead, full feature parity.

**Cons:** None, really this is the ideal environment.

### 3.2 macOS (via Docker Desktop)

macOS doesn't have a Linux kernel, so Docker Desktop runs a **lightweight Linux VM** behind the scenes using Apple's Virtualization framework (or HyperKit on older versions).

**Pros:** Seamless developer experience, GUI for managing containers.

**Cons:** Slight performance overhead due to the VM layer. File system sharing between Mac and container can be slow.

### 3.3 Windows (Two Options)

1. **Docker Desktop with WSL 2 backend** (*recommended*) — Uses the Windows Subsystem for Linux 2, which runs a real Linux kernel inside Windows. Docker integrates directly with this.
2. **Docker Desktop with Hyper-V backend** (*legacy*) — Uses a full Hyper-V VM. Heavier and slower than WSL 2.

### 3.4 WSL 2 (Windows Subsystem for Linux)

WSL 2 deserves its own mention because it's the **best way to run Docker on Windows**.

```
# Enable WSL 2 (from PowerShell as Admin)
wsl --install

# Install a Linux distribution (e.g., Ubuntu)
wsl --install -d Ubuntu

# Inside WSL 2, Docker CLI works natively
docker run hello-world
```



**Recommendation for Windows users:** Install WSL 2 + Docker Desktop with the WSL 2 backend. You get near-native Linux performance with full Windows integration.

## 4. Background Knowledge Before Learning Docker

Before diving deeper, make sure you're comfortable with these foundational concepts:

### 4.1 Linux Basics

1. **File system navigation** — `cd`, `ls`, `pwd`, `cat`, `mkdir`, `rm`
2. **File permissions** — `chmod`, `chown`, understanding `rwX`
3. **Process management** — `ps`, `top`, `kill`, understanding PID
4. **Package management** — `apt`, `yum`, `apk` (used inside containers)
5. **Environment variables** — `export`, `echo $VAR`, `.env` files

### 4.2 Networking Fundamentals

1. **IP addresses and ports** — Every service listens on a port (e.g., HTTP on port 80)
2. **TCP vs UDP** — Most web traffic is TCP
3. **DNS** — How domain names resolve to IP addresses
4. **localhost (127.0.0.1)** — The loopback address pointing to your own machine

## 4.3 Client-Server Architecture

1. A **server** listens for requests on a port
2. A **client** sends requests to that port
3. Docker itself follows this model — the CLI (client) talks to the daemon (server)

## 4.4 YAML Basics

Docker Compose uses YAML files. Know the basics:

```
# YAML uses indentation (spaces, NOT tabs)
services:
  web:
    image: nginx
    ports:
      - "8080:80"    # host_port:container_port
    environment:
      - APP_ENV=production
```

# 5. Docker Terminology — The Complete Glossary

This is the section you'll keep coming back to. Every term is explained with context and examples.

## 5.1 Docker Image

A **read-only template** used to create containers. Think of it as a snapshot or a blueprint.

- Images are built in **layers**. Each instruction in a Dockerfile creates a new layer.
- Layers are cached — if nothing changed, Docker reuses the cached layer (faster builds).
- Images are stored in **registries** (like Docker Hub).

```
# Pull an image from Docker Hub
docker pull python:3.11-slim

# List all images on your system
docker images

# Remove an image
docker rmi python:3.11-slim
```

## 5.2 Docker Container

A **running instance** of an image. If an image is the blueprint, a container is the actual building.

- Containers are **ephemeral** by default — when they stop, any data written inside them is lost (unless you use volumes).
- Each container gets its own filesystem, network, and process space.

```
# Create and start a container
docker run -d --name my-app python:3.11-slim sleep 3600

# List running containers
docker ps

# List ALL containers (including stopped)
docker ps -a
```

```
# Stop a container
docker stop my-app
```

```
# Remove a container
docker rm my-app
```



**Image vs Container:** An image is like a class in programming. A container is like an object (instance) of that class. You can create many containers from the same image.

## 5.3 Dockerfile

A **text file with instructions** for building a Docker image. Each line is a step.

```
# Start from a base image
FROM python:3.11-slim

# Set the working directory inside the container
WORKDIR /app

# Copy requirements file first (for layer caching)
COPY requirements.txt .

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code
COPY . .

# Expose port 8000 (documentation purpose)
EXPOSE 8000
```



```
# Define the command to run when the container starts
CMD ["python", "server.py"]
```

## Common Dockerfile Instructions

| Instruction | Purpose                                   | Example                 |
|-------------|-------------------------------------------|-------------------------|
| FROM        | Base image to build from                  | FROM node:18-alpine     |
| WORKDIR     | Set working directory                     | WORKDIR /app            |
| COPY        | Copy files from host to image             | COPY . .                |
| RUN         | Execute a command during build            | RUN apt-get update      |
| EXPOSE      | Document which port the app uses          | EXPOSE 3000             |
| ENV         | Set environment variables                 | ENV NODE_ENV=production |
| CMD         | Default command when container starts     | CMD ["node", "app.js"]  |
| ENTRYPOINT  | Fixed command (CMD becomes arguments)     | ENTRYPOINT ["python"]   |
| ARG         | Build-time variables                      | ARG VERSION=1.0         |
| VOLUME      | Declare a mount point for persistent data | VOLUME /data            |

## 5.4 Docker Daemon ( `dockerd` )

The **background service** that manages everything. It:

- Builds images from Dockerfiles
- Pulls/pushes images to registries
- Creates, starts, stops, and removes containers
- Manages networks and volumes
- Exposes a REST API on a **Docker Socket**

## 5.5 Docker Socket ( `/var/run/docker.sock` )

The **Unix socket** through which the Docker CLI communicates with the Docker Daemon.

```
# The socket is a file on your system
ls -la /var/run/docker.sock

# You can even talk to Docker via curl through this socket
curl --unix-socket /var/run/docker.sock http://localhost/v1.41/containers/json
```



**Security warning:** Mounting the Docker socket inside a container ( `-v /var/run/docker.sock:/var/run/docker.sock` ) gives that container **full control** over the Docker daemon. This is powerful but dangerous — treat it like giving root access.

## 5.6 Docker Client

The **command-line interface** ( `docker` ). It sends commands to the daemon.

- The client and daemon don't have to be on the same machine. You can configure the client to talk to a remote daemon via TCP.

```
# Default: client talks to local daemon via socket
docker ps

# Talk to a remote daemon
docker -H tcp://192.168.1.100:2376 ps
```

## 5.7 Docker Registry

A **storage and distribution service** for Docker images. Think of this like github for docker images, store, version and pull from this hub.

1. **Docker Hub** — The default public registry. Free for public images.
2. **Amazon ECR** — AWS's private registry.
3. **GitHub Container Registry ([ghcr.io](https://github.com/docker/registry))** — GitHub's registry.

4. **Self-hosted** — You can run your own registry with `docker run -d -p 5000:5000 registry:2`

```
# Tag an image for a registry
docker tag my-app:latest myusername/my-app:v1.0

# Push to Docker Hub
docker push myusername/my-app:v1.0

# Pull from Docker Hub
docker pull myusername/my-app:v1.0
```

## 5.8 Docker Plugins

Extend Docker's functionality. Common types:

1. **Volume plugins** — Store data on external systems (e.g., NFS, AWS EBS)
2. **Network plugins** — Custom networking (e.g., Weave, Calico)
3. **Log plugins** — Send logs to external systems (e.g., Splunk, Fluentd)

```
# List installed plugins
docker plugin ls

# Install a plugin
docker plugin install vieux/sshfs
```

## 5.9 Docker Metrics and Logs

### Metrics

```
# Real-time resource usage of all running containers
docker stats

# Output:
# CONTAINER ID   NAME      CPU %   MEM USAGE / LIMIT   MEM %   NET
```

```
I/O  BLOCK I/O
# a1b2c3d4e5f6  my-app  0.50%  50MiB / 512MiB  9.77%  1.2
kB    0B
```

## Logs

```
# View logs from a container
docker logs my-app

# Follow logs in real-time (like tail -f)
docker logs -f my-app

# Show last 100 lines with timestamps
docker logs --tail 100 -t my-app
```

## 5.10 Interactive Shell ( `-it` flag)

Get a terminal **inside** a running container incredibly useful for debugging and running code directly, for eg, in a database container like postgres you can run sql queries.

```
# Start a container with an interactive shell
docker run -it ubuntu:22.04 /bin/bash

# Exec into an already-running container
docker exec -it my-app /bin/bash

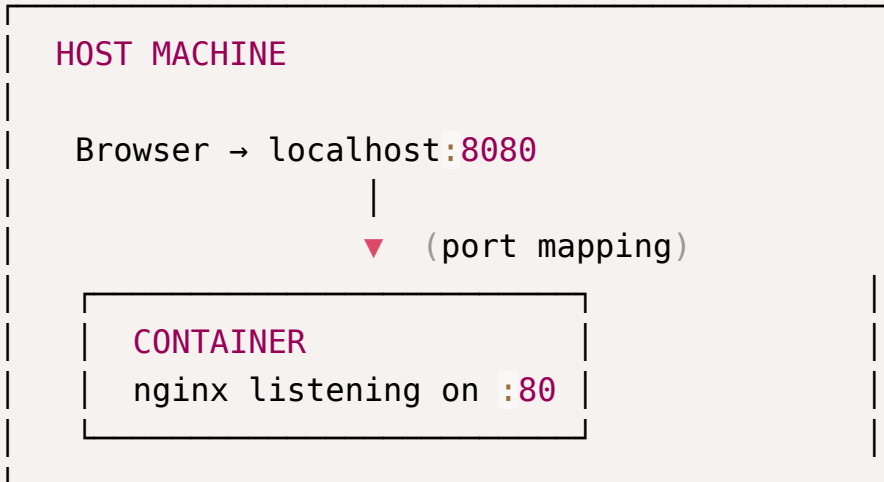
# What -i and -t mean:
# -i (--interactive) → Keep STDIN open
# -t (--tty) → Allocate a pseudo-terminal
```

## 5.11 Ports and Port Mapping

Containers are **isolated by default** they have their own network. To access a service inside a container from your host machine, you need **port mapping**.

```
# Syntax: -p HOST_PORT:CONTAINER_PORT
docker run -d -p 8080:80 nginx
```

```
# Now: localhost:8080 on your machine → port 80 inside the container
```



## Mapping multiple ports

```
# Map multiple ports
docker run -d -p 8080:80 -p 8443:443 nginx

# Map to a specific host interface
docker run -d -p 127.0.0.1:8080:80 nginx

# Let Docker assign a random host port
docker run -d -p 80 nginx
docker port <container_id> # See the assigned port
```

## 5.12 Docker Volumes

Volumes solve the problem of **data persistence**. Remember, containers are ephemeral when they're removed, their data is gone. Volumes exist **outside** the container lifecycle.

## Three types of data storage:

1. **Named Volumes** (*recommended*) — Managed by Docker, stored in Docker's storage area.
2. **Bind Mounts** — Map a specific host directory into the container.
3. **tmpfs Mounts** — Store data in the host's memory only (not persisted to disk).

```
# Create a named volume
docker volume create my-data
```

```
# Run a container with a named volume
docker run -d -v my-data:/app/data my-app
```

```
# Run with a bind mount (host directory → container directory)
docker run -d -v $(pwd)/local-data:/app/data my-app
```

```
# List all volumes
docker volume ls
```

```
# Inspect a volume
docker volume inspect my-data
```

```
# Remove unused volumes
docker volume prune
```



### When to use which?

- **Named volumes** → Database storage, persistent app data
- **Bind mounts** → Local development (edit code on host, see changes in container)
- **tmpfs** → Sensitive data you don't want on disk (e.g., secrets during build)

## 5.13 Docker Networking — Ingress and Egress

### What is Ingress and Egress?

- **Ingress** = Traffic coming *into* the container (e.g., a user hitting your web server)
- **Egress** = Traffic going *out* of the container (e.g., your app calling an external API)

### Docker Network Types

```
# List all networks
docker network ls
```

1. **bridge** (*default*) — Containers on the same bridge network can talk to each other by container name. Isolated from the host network.
2. **host** — Container shares the host's network stack directly. No isolation, but no overhead.
3. **none** — Container has no network access at all.
4. **overlay** — Spans multiple Docker hosts. Used in Docker Swarm / orchestration.

```
# Create a custom bridge network
docker network create my-network
```

```
# Run containers on the same network
docker run -d --name api --network my-network my-api
docker run -d --name db --network my-network postgres:15

# Now 'api' container can reach 'db' container by name:
# postgres://db:5432/mydb
```



**Pro tip:** Always create custom networks instead of using the default bridge. Custom networks provide **automatic DNS resolution** — containers can find each other by name. The default bridge does not.

## 5.14 Restarting and Rebuilding Containers

### Restarting

```
# Restart a running container
docker restart my-app

# Restart policies (auto-restart on failure)
docker run -d --restart=always my-app           # Always restart
docker run -d --restart=on-failure:5 my-app      # Restart up to 5 times on failure
docker run -d --restart=unless-stopped my-app    # Restart unless manually stopped
```

### Rebuilding

When you change your code or Dockerfile, you need to **rebuild** the image and recreate the container:

```
# Step 1: Rebuild the image
docker build -t my-app:v2 .
```



```
# Step 2: Stop and remove the old container
docker stop my-app && docker rm my-app

# Step 3: Run the new container
docker run -d --name my-app -p 8080:8000 my-app:v2
```

## 6. Hands-On Project : Python Task Manager API

Let's tie everything together with a **real example**. We'll build a simple Python Flask API, containerize it, and demonstrate every Docker concept we've covered.

### 6.1 The Application Code

#### Project Structure

```
task-manager/
├── server.py
├── requirements.txt
├── Dockerfile
├── docker-compose.yml
├── .env
└── .dockerignore
```

#### server.py

```
import os
import json
from flask import Flask, request, jsonify
from datetime import datetime

app = Flask(__name__)
```

```

# In-memory storage (in production, use a database)
tasks = []
task_id_counter = 1

@app.route("/health", methods=["GET"])
def health_check():
    return jsonify({
        "status": "healthy",
        "timestamp": datetime.utcnow().isoformat(),
        "environment": os.getenv("APP_ENV", "development")
    })

@app.route("/tasks", methods=["GET"])
def get_tasks():
    return jsonify({"tasks": tasks, "count": len(tasks)})

@app.route("/tasks", methods=["POST"])
def create_task():
    global task_id_counter
    data = request.get_json()

    if not data or "title" not in data:
        return jsonify({"error": "Title is required"}), 400

    task = {
        "id": task_id_counter,
        "title": data["title"],
        "description": data.get("description", ""),
        "completed": False,
        "created_at": datetime.utcnow().isoformat()
    }
    tasks.append(task)
    task_id_counter += 1
    return jsonify(task), 201

@app.route("/tasks/<int:task_id>", methods=["PUT"])

```

```

def update_task(task_id):
    task = next((t for t in tasks if t["id"] == task_id), None)
    if not task:
        return jsonify({"error": "Task not found"}), 404

    data = request.get_json()
    task["title"] = data.get("title", task["title"])
    task["description"] = data.get("description", task["description"])
    task["completed"] = data.get("completed", task["completed"])
    return jsonify(task)

@app.route("/tasks/<int:task_id>", methods=["DELETE"])
def delete_task(task_id):
    global tasks
    tasks = [t for t in tasks if t["id"] != task_id]
    return jsonify({"message": "Task deleted"}), 200

if __name__ == "__main__":
    port = int(os.getenv("PORT", 8000))
    debug = os.getenv("APP_ENV", "development") == "development"
    app.run(host="0.0.0.0", port=port, debug=debug)

```

### requirements.txt

```

flask==3.0.0
gunicorn==21.2.0

```

### .env

```

APP_ENV=development
PORT=8000

```

```
SECRET_KEY=my-secret-key-change-in-production
```

### **.dockerignore**

```
__pycache__  
*.pyc  
.git  
.env  
.venv  
node_modules
```

## 6.2 The Dockerfile

```
# — Stage 1: Base image —  
FROM python:3.11-slim AS base  
  
# Set environment variables  
ENV PYTHONDONTWRITEBYTECODE=1 \  
    PYTHONUNBUFFERED=1  
  
# Set working directory  
WORKDIR /app  
  
# — Stage 2: Install dependencies —  
FROM base AS dependencies  
  
# Copy only requirements first (layer caching optimization)  
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt  
  
# — Stage 3: Final image —  
FROM dependencies AS final  
  
# Copy application code
```

```

COPY . .

# Expose the application port (documentation)
EXPOSE 8000

# Health check – Docker will periodically check if the container is healthy
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# Run with gunicorn in production, flask dev server in development
CMD ["gunicorn", "--bind", "0.0.0.0:8000", "--workers", "4", "server:app"]

```

## 6.3 Every Docker Command : Building the Project

Follow these steps in order. Each demonstrates a core Docker concept.

### Step 1: Build the image

```

# Build and tag the image
docker build -t task-manager:v1 .

# What happens:
# 1. Docker reads the Dockerfile
# 2. Each instruction creates a layer
# 3. Layers are cached for future builds
# 4. The final image is tagged as task-manager:v1

```

### Step 2: Run the container

Run this in your terminal.

```

# Run in detached mode with port mapping and env variables
docker run -d \

```

```
--name task-api \  
-p 8080:8000 \  
-e APP_ENV=development \  
-e SECRET_KEY=dev-secret \  
--restart=unless-stopped \  
task-manager:v1
```

# Breakdown:

```
# -d                → Run in background (detached)  
# --name task-api   → Give it a human-readable name  
# -p 8080:8000      → Map host port 8080 → container port  
8000  
# -e APP_ENV=...    → Set environment variables  
# --restart=unless... → Auto-restart policy
```

### Step 3: Interact with the running container

```
# Check if it's running  
docker ps
```

```
# View logs  
docker logs task-api  
docker logs -f task-api # Follow in real-time
```

```
# Check resource usage  
docker stats task-api
```

```
# Get a shell inside the container  
docker exec -it task-api /bin/bash
```

```
# Test the API  
curl http://localhost:8080/health  
curl -X POST http://localhost:8080/tasks \  
-H "Content-Type: application/json" \  
-d '{"title": "Learn Docker", "description": "Complete the
```

```
tutorial"}'  
curl http://localhost:8080/tasks
```

## Step 4: Volumes for persistent data

```
# Create a volume for data  
docker volume create task-data  
  
# Run with the volume mounted  
docker run -d \  
  --name task-api \  
  -p 8080:8000 \  
  -v task-data:/app/data \  
  task-manager:v1  
  
# For development – bind mount your source code  
docker run -d \  
  --name task-api-dev \  
  -p 8080:8000 \  
  -v $(pwd):/app \  
  task-manager:v1
```

## Step 5: Networking with multiple containers

```
# Create a custom network  
docker network create task-network  
  
# Run a PostgreSQL database  
docker run -d \  
  --name task-db \  
  --network task-network \  
  -e POSTGRES_DB=tasks \  
  -e POSTGRES_USER=admin \  
  -e POSTGRES_PASSWORD=secret \  
  -v pgdata:/var/lib/postgresql/data
```

```
postgres:15

# Run the API on the same network
docker run -d \
  --name task-api \
  --network task-network \
  -p 8080:8000 \
  -e DATABASE_URL=postgresql://admin:secret@task-db:5432/task
s \
  task-manager:v1

# The API can now reach the database at "task-db:5432"
# Docker's DNS resolves the container name automatically
```

## Step 6: Cleanup commands

```
# Stop all running containers
docker stop $(docker ps -q)

# Remove all stopped containers
docker container prune

# Remove unused images
docker image prune

# Remove unused volumes
docker volume prune

# Nuclear option – remove EVERYTHING unused
docker system prune -a --volumes
```

# 7. Docker Compose : Multi-Container Made Easy



Managing multiple `docker run` commands gets tedious fast. **Docker Compose** lets you define your entire application stack in a single YAML file.

## 7.1 `docker-compose.yml`

```
version: "3.9"

services:
  # — The API Server —
  api:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: task-api
    ports:
      - "8080:8000"
    environment:
      - APP_ENV=development
      - PORT=8000
      - DATABASE_URL=postgresql://admin:secret@db:5432/tasks
      - REDIS_URL=redis://cache:6379
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    networks:
      - app-network
    restart: unless-stopped

  # — PostgreSQL Database —
  db:
    image: postgres:15-alpine
    container_name: task-db
    environment:
```

```

    POSTGRES_DB: tasks
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: secret
volumes:
  - pgdata:/var/lib/postgresql/data
ports:
  - "5432:5432"
networks:
  - app-network
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U admin -d tasks"]
  interval: 10s
  timeout: 5s
  retries: 5
restart: unless-stopped

# — Redis Cache —
cache:
  image: redis:7-alpine
  container_name: task-cache
  ports:
    - "6379:6379"
  volumes:
    - redisdata:/data
  networks:
    - app-network
  restart: unless-stopped

# — Shared Resources —
networks:
  app-network:
    driver: bridge

volumes:

```

```
pgdata:  
redisdata:
```

## 7.2 Docker Compose Commands

```
# Start all services (build images if needed)  
docker compose up -d --build
```

```
# View status of all services  
docker compose ps
```

```
# View logs from all services  
docker compose logs -f
```

```
# View logs from a specific service  
docker compose logs -f api
```

```
# Stop all services  
docker compose down
```

```
# Stop and remove volumes (delete all data)  
docker compose down -v
```

```
# Rebuild a specific service  
docker compose build api
```

```
# Scale a service (run 3 instances of the API)  
docker compose up -d --scale api=3
```

```
# Execute a command in a running service  
docker compose exec api /bin/bash
```



**Compose vs `docker run`** : Compose is *declarative* — you describe what you want, and Docker makes it happen. `docker run` is *imperative* — you specify every step manually. For anything more than a single container, **always use Compose**.

## 8. Local Development Setup : Complete Walkthrough

### 8.1 Docker Desktop Setup

1. **Download** Docker Desktop from [docker.com](https://docker.com)
2. **Install** and follow the platform-specific instructions
3. **Verify** the installation:

```
docker --version      # Docker CLI version
docker compose version # Compose version
docker info           # Full system info
```

### 8.2 Docker CLI in WSL 2 (Windows)

```
# Step 1: Open WSL terminal
wsl

# Step 2: Docker Desktop integrates with WSL automatically
# Verify:
docker --version

# Step 3: Navigate to your project
cd /mnt/c/Users/YourName/projects/task-manager
```

```
# Step 4: Build and run
docker compose up -d --build
```

## 8.3 Building Images Efficiently

```
# Standard build
docker build -t my-app .

# Build with no cache (force fresh build)
docker build --no-cache -t my-app .

# Build with build arguments
docker build --build-arg VERSION=2.0 -t my-app .

# Build for a specific platform
docker build --platform linux/amd64 -t my-app .

# Multi-platform build (for deploying to different architectures)
docker buildx build --platform linux/amd64,linux/arm64 -t my-app .
```

## 8.4 Environment Variables — Best Practices

```
# Method 1: Inline with -e flag
docker run -e APP_ENV=production my-app

# Method 2: From an .env file
docker run --env-file .env my-app

# Method 3: In docker-compose.yml (see section 7.1)
```

```
# Method 4: In the Dockerfile (baked into the image)
# ENV APP_ENV=production ← Avoid for secrets!
```



**Never bake secrets into images.** Anyone who pulls your image can extract the environment variables. Use `--env-file`, Docker Secrets, or a secrets manager (like AWS Secrets Manager) for sensitive data.

## 8.5 Running Minikube (Local Kubernetes)

Minikube lets you run a **single-node Kubernetes cluster** locally ,useful for testing K8s deployments before going to production.

Inside your wsl terminal

```
# Install Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start a cluster (uses Docker as the driver)
minikube start --driver=docker

# Check status
minikube status

# Use kubectl to interact with the cluster
kubectl get nodes
kubectl get pods --all-namespaces

# Deploy your Docker image to Minikube
# First, point your shell to Minikube's Docker daemon
eval $(minikube docker-env)

# Build the image (now it's available inside Minikube)
docker build -t task-manager:v1 .
```

```
# Create a Kubernetes deployment
kubectl create deployment task-api --image=task-manager:v1

# Expose it as a service
kubectl expose deployment task-api --type=NodePort --port=8000

# Get the URL
minikube service task-api --url

# Stop Minikube
minikube stop
```

## 9. Production Deployment on AWS

Now we move from local development to **production-grade deployment**. We'll cover three approaches, from simplest to most scalable.

### 9.1 Option A: Deploy to EC2 (Simplest)

An **EC2 instance** is a virtual server on AWS. You SSH in, install Docker, and run your containers. Simple but manual.

#### Step-by-step

```
# 1. Launch an EC2 instance (Amazon Linux 2023 or Ubuntu)
# 2. SSH into the instance
ssh -i my-key.pem ec2-user@<your-ec2-ip>

# 3. Install Docker
sudo yum update -y
sudo yum install -y docker
sudo systemctl start docker
sudo systemctl enable docker
```

```

sudo usermod -aG docker ec2-user

# 4. Install Docker Compose
sudo curl -L "https://github.com/docker/compose/releases/latest/download/docker-compose-$(uname -s)-$(uname -m)" \
  -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose

# 5. Clone your project
git clone https://github.com/youruser/task-manager.git
cd task-manager

# 6. Run with Compose
docker compose up -d --build

```

**Pros:** Simple, full control, cheap for small projects.

**Cons:** No auto-scaling, no load balancing, manual updates, single point of failure.

## 9.2 Option B: Deploy to ECS (Elastic Container Service)

**ECS** is AWS's managed container orchestration service. It handles container placement, scaling, and load balancing for you. You don't worry about the resources you just pull images from ECR or other image hub and write a ECS script that runs the container.

### Key ECS Concepts

| Concept                | What it is                                                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>Cluster</b>         | A logical grouping of tasks and services                                                                     |
| <b>Task Definition</b> | A blueprint for your container (like a docker-compose service). Defines image, CPU, memory, ports, env vars. |
| <b>Task</b>            | A running instance of a Task Definition                                                                      |
| <b>Service</b>         | Ensures a desired number of tasks are always running. Integrates with load balancers.                        |



| Concept | What it is                                                                                       |
|---------|--------------------------------------------------------------------------------------------------|
| Fargate | Serverless compute for ECS. No EC2 instances to manage you just define the task and AWS runs it. |

## Deployment Steps

Make sure that you have configure aws cli in your terminal. Get your aws secrets from IAM for a user that has necessary permission or you can adjust the IAM policies based on your needs.

```
# 1. Create an ECR (Elastic Container Registry) repository
aws ecr create-repository --repository-name task-manager

# 2. Authenticate Docker with ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
    <account-id>.dkr.ecr.us-east-1.amazonaws.com

# 3. Tag and push your image
docker tag task-manager:v1 \
  <account-id>.dkr.ecr.us-east-1.amazonaws.com/task-manager:v
1

docker push \
  <account-id>.dkr.ecr.us-east-1.amazonaws.com/task-manager:v
1

# 4. Create a Task Definition (JSON)
```

## ECS Task Definition ( `task-definition.json` )

```
{
  "family": "task-manager",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
```

```

    "memory": "512",
    "executionRoleArn": "arn:aws:iam::<account-id>:role/ecsTask
ExecutionRole",
    "containerDefinitions": [
      {
        "name": "task-api",
        "image": "<account-id>.dkr.ecr.us-east-1.amazonaws.com/
task-manager:v1",
        "portMappings": [
          {
            "containerPort": 8000,
            "protocol": "tcp"
          }
        ],
        "environment": [
          { "name": "APP_ENV", "value": "production" },
          { "name": "PORT", "value": "8000" }
        ],
        "logConfiguration": {
          "logDriver": "awslogs",
          "options": {
            "awslogs-group": "/ecs/task-manager",
            "awslogs-region": "us-east-1",
            "awslogs-stream-prefix": "ecs"
          }
        },
        "healthCheck": {
          "command": ["CMD-SHELL", "curl -f http://localhost:80
00/health || exit 1"],
          "interval": 30,
          "timeout": 5,
          "retries": 3
        }
      }
    ]
  }
}

```

```
]
}
```

# 5. Register the task definition

```
aws ecs register-task-definition --cli-input-json file://task-
definition.json
```

# 6. Create a cluster

```
aws ecs create-cluster --cluster-name task-manager-cluster
```

# 7. Create a service with load balancer

```
aws ecs create-service \
  --cluster task-manager-cluster \
  --service-name task-api-service \
  --task-definition task-manager:1 \
  --desired-count 2 \
  --launch-type FARGATE \
  --network-configuration "awsvpcConfiguration={subnets=[subn
et-xxx],securityGroups=[sg-xxx],assignPublicIp=ENABLED}" \
  --load-balancers "targetGroupArn=arn:aws:...,containerName=
task-api,containerPort=8000"
```

## 9.3 Option C: Deploy to EKS (Elastic Kubernetes Service)

**EKS** is AWS's managed Kubernetes service. It's the most powerful and complex option, ideal for large-scale, multi-service architectures.

### Kubernetes Manifest ( `k8s/deployment.yaml` )

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: task-api
```

```

labels:
  app: task-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: task-api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
  template:
    metadata:
      labels:
        app: task-api
    spec:
      containers:
        - name: task-api
          image: <account-id>.dkr.ecr.us-east-1.amazonaws.com/task-manager:v1
          ports:
            - containerPort: 8000
          env:
            - name: APP_ENV
              value: "production"
            - name: PORT
              value: "8000"
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          livenessProbe:

```

```
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 10
        periodSeconds: 30
      readinessProbe:
        httpGet:
          path: /health
          port: 8000
        initialDelaySeconds: 5
        periodSeconds: 10
    ---
apiVersion: v1
kind: Service
metadata:
  name: task-api-service
spec:
  type: LoadBalancer
  selector:
    app: task-api
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
```

```
# Deploy to EKS
kubectl apply -f k8s/deployment.yaml

# Check deployment status
kubectl get deployments
kubectl get pods
kubectl get services
```

## 10. CI/CD : Automated Build, Test, and Deploy

The final piece: **automating everything** so that every code push automatically builds a new image, tests it, and deploys it with **zero downtime**.

### 10.1 GitHub Actions Pipeline

( [.github/workflows/deploy.yml](#) )

```
name: Build and Deploy to ECS

on:
  push:
    branches: [main]

env:
  AWS_REGION: us-east-1
  ECR_REPOSITORY: task-manager
  ECS_CLUSTER: task-manager-cluster
  ECS_SERVICE: task-api-service
  CONTAINER_NAME: task-api

jobs:
  # — JOB 1: Test —
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"
```

```

- name: Install dependencies
  run: pip install -r requirements.txt -r requirements-
dev.txt

- name: Run tests
  run: pytest tests/ -v --cov=.

# — JOB 2: Build and Push Image —
build:
  needs: test
  runs-on: ubuntu-latest
  outputs:
    image: $ steps.build-image.outputs.image
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: $ secrets.AWS_ACCESS_KEY_ID
        aws-secret-access-key: $ secrets.AWS_SECRET_ACCESS_
KEY
        aws-region: $ env.AWS_REGION

    - name: Login to Amazon ECR
      id: login-ecr
      uses: aws-actions/amazon-ecr-login@v2

    - name: Build, tag, and push image
      id: build-image
      env:
        ECR_REGISTRY: $ steps.login-ecr.outputs.registry
        IMAGE_TAG: $ github.sha
      run: |
        docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:$IMAG

```

```

E_TAG .
    docker build -t $ECR_REGISTRY/$ECR_REPOSITORY:lates
t .
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TA
G
    docker push $ECR_REGISTRY/$ECR_REPOSITORY:latest
    echo "image=$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TA
G" >> $GITHUB_OUTPUT

```

# — JOB 3: Deploy with Zero Downtime —

deploy:

needs: build

runs-on: ubuntu-latest

steps:

- name: Checkout code
 uses: actions/checkout@v4
- name: Configure AWS credentials
 uses: aws-actions/configure-aws-credentials@v4
 with:
 aws-access-key-id: \$ secrets.AWS\_ACCESS\_KEY\_ID
 aws-secret-access-key: \$ secrets.AWS\_SECRET\_ACCESS\_

KEY

aws-region: \$ env.AWS\_REGION

- name: Download current task definition
 run: |
 aws ecs describe-task-definition \
 --task-definition task-manager \
 --query taskDefinition > task-definition.json
- name: Update task definition with new image
 id: task-def
 uses: aws-actions/amazon-ecs-render-task-definition@v

1

with:



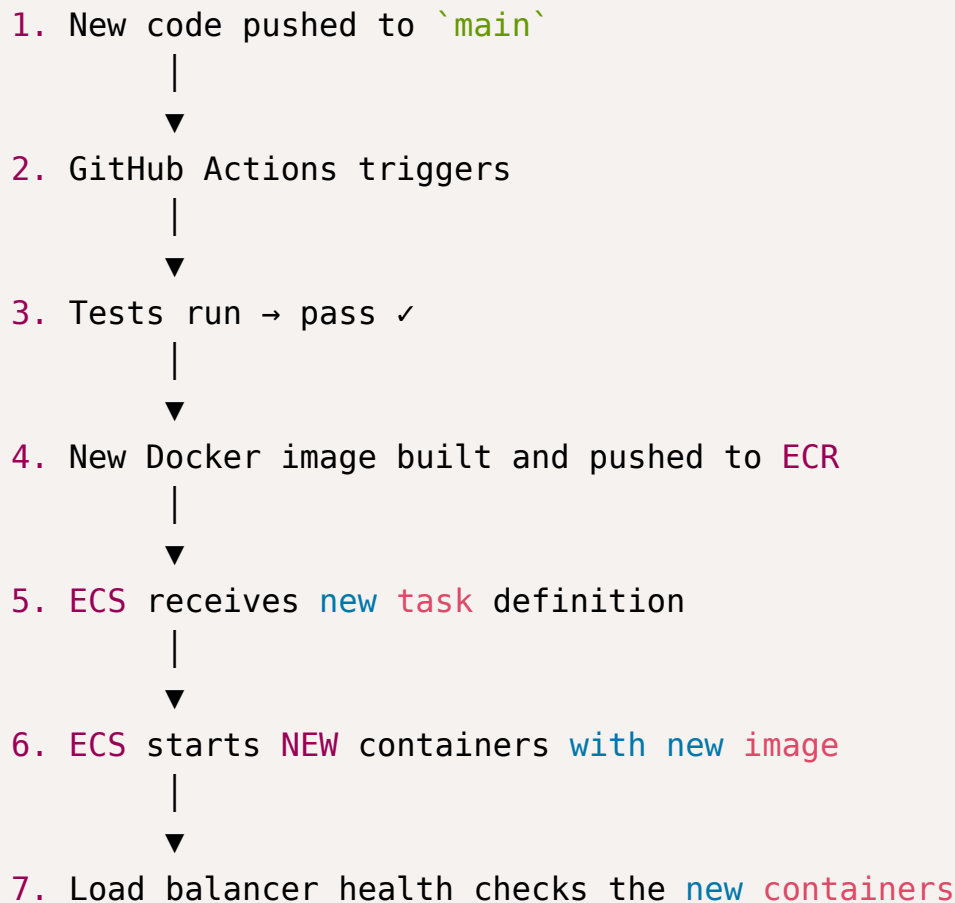
```

        task-definition: task-definition.json
        container-name: $ env.CONTAINER_NAME
        image: $ needs.build.outputs.image

- name: Deploy to ECS (rolling update = zero downtime)
  uses: aws-actions/amazon-ecs-deploy-task-definition@v
2
  with:
    task-definition: $ steps.task-def.outputs.task-defi
    nition
    service: $ env.ECS_SERVICE
    cluster: $ env.ECS_CLUSTER
    wait-for-service-stability: true

```

## 10.2 How Zero-Downtime Deployment Works



▼  
8. Once healthy → traffic shifts to **new containers**

▼  
9. **OLD** containers are gracefully stopped

▼  
10. Zero downtime achieved!



**The key to zero downtime** is the **rolling update strategy**. ECS (or Kubernetes) never kills old containers until new ones pass health checks. Users are always hitting healthy containers through the load balancer.

## 11. Quick Reference : Essential Docker Commands

These are essential commands that you will use frequently, make sure to familiarize yourself not by memorizing but by practicing in real apps.

| Category   | Command                                   | What it does                |
|------------|-------------------------------------------|-----------------------------|
| Images     | <code>docker build -t name .</code>       | Build image from Dockerfile |
|            | <code>docker images</code>                | List all images             |
|            | <code>docker rmi image</code>             | Remove an image             |
|            | <code>docker pull image</code>            | Download from registry      |
|            | <code>docker push image</code>            | Upload to registry          |
| Containers | <code>docker run -d --name n image</code> | Run a container             |
|            | <code>docker ps</code>                    | List running containers     |
|            | <code>docker stop name</code>             | Stop a container            |
|            | <code>docker rm name</code>               | Remove a container          |

| Category        | Command                                | What it does             |
|-----------------|----------------------------------------|--------------------------|
|                 | <code>docker logs -f name</code>       | Follow container logs    |
|                 | <code>docker exec -it name bash</code> | Shell into container     |
| <b>Volumes</b>  | <code>docker volume create v</code>    | Create a volume          |
|                 | <code>docker volume ls</code>          | List volumes             |
|                 | <code>docker volume prune</code>       | Remove unused volumes    |
| <b>Networks</b> | <code>docker network create n</code>   | Create a network         |
|                 | <code>docker network ls</code>         | List networks            |
| <b>Compose</b>  | <code>docker compose up -d</code>      | Start all services       |
|                 | <code>docker compose down</code>       | Stop all services        |
|                 | <code>docker compose logs -f</code>    | Follow all logs          |
| <b>Cleanup</b>  | <code>docker system prune -a</code>    | Remove everything unused |

## 12. What's Next?

Now that you have a solid foundation, here's a roadmap for going deeper. Each topic builds on what you've already learned. This part is not for beginners so do not bother wasting time if you are not confident with the fundamentals.

### 12.1 Docker Security

Containers share the host kernel, so a misconfigured container can be a real security risk. Here's how to lock things down.

#### Run Containers as Non-Root

By default, containers run as **root** this is dangerous. If an attacker escapes the container, they have root access to the host.

```
# In your Dockerfile, create and switch to a non-root user
FROM python:3.11-slim
```

```
RUN groupadd -r appuser && useradd -r -g appuser appuser
```

```
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir -r requirements.txt

# Switch to non-root user
USER appuser

CMD ["python", "server.py"]
```

## Scan Images for Vulnerabilities

**Trivy** is an open-source scanner that detects known vulnerabilities (CVEs) in your images.

```
# Install Trivy
sudo apt-get install -y trivy

# Scan an image
trivy image task-manager:v1

# Scan and fail if critical vulnerabilities are found (useful
in CI/CD)
trivy image --exit-code 1 --severity CRITICAL task-manager:v1
```

## Secrets Management

Never hardcode secrets. Use Docker's built-in secrets or external managers.

```
# Docker Swarm secrets (encrypted at rest)
echo "my-db-password" | docker secret create db_password -

# In docker-compose.yml (Swarm mode)
services:
  api:
    image: task-manager:v1
    secrets:
```

```
- db_password
```

```
secrets:  
  db_password:  
    external: true
```

For **non-Swarm** setups, use:

- **AWS Secrets Manager** or **SSM Parameter Store** for production
- `--env-file` with restricted file permissions for development
- **HashiCorp Vault** for advanced multi-cloud secret management



**Security checklist:** (1) Non-root user in Dockerfile. (2) Scan images in CI/CD. (3) Never bake secrets into images. (4) Use read-only filesystems where possible ( `--read-only` flag). (5) Drop unnecessary Linux capabilities ( `--cap-drop ALL` ).

## 12.2 Multi-Stage Builds

Your build tools (compilers, dev dependencies, test frameworks) don't belong in your production image. **Multi-stage builds** let you separate the build environment from the runtime environment, resulting in dramatically smaller and more secure images.

### The Problem

```
# A typical Node.js image with build tools  
# node:18                → ~900 MB  
# node:18-alpine         → ~130 MB  
# Your app alone         → ~50 MB ← this is all you need in produc  
tion
```

### The Solution

```

# — Stage 1: Build —
FROM node:18 AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build          # Compile TypeScript, bundle assets,
                             etc.
RUN npm prune --production # Remove dev dependencies

# — Stage 2: Production —
FROM node:18-alpine AS production

WORKDIR /app

# Copy ONLY what's needed from the build stage
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./

USER node
EXPOSE 3000
CMD ["node", "dist/index.js"]

```

## Another Example: Go Application

Go compiles to a single binary, so the production image can be **incredibly small**.

```

# — Build stage —
FROM golang:1.21 AS builder
WORKDIR /app
COPY go.* ./
RUN go mod download
COPY . .

```

```
RUN CGO_ENABLED=0 GOOS=linux go build -o server .
```

```
# — Production stage —
```

```
FROM scratch
```

```
COPY --from=builder /app/server /server
```

```
EXPOSE 8080
```

```
CMD ["/server"]
```

```
# Final image size: ~10-15 MB (compared to ~1 GB with the Go SDK)
```



**Rule of thumb:** If your final image contains compilers, package managers, or test frameworks, you should be using multi-stage builds. Your production image should only have what's needed to *run* the application.

## 12.3 Container Orchestration : Docker Swarm vs Kubernetes

When you have many containers across many servers, you need an **orchestrator** to manage them. The two main options are Docker Swarm and Kubernetes.

### Docker Swarm

Swarm is Docker's **built-in** orchestration tool. It turns a group of Docker hosts into a single virtual host.

```
# Initialize a Swarm (on the manager node)
```

```
docker swarm init --advertise-addr <manager-ip>
```

```
# Join worker nodes to the Swarm
```

```
# (run this on each worker machine)
```

```
docker swarm join --token <token> <manager-ip>:2377
```

```
# Deploy a stack (like Compose, but distributed)
```

```

docker stack deploy -c docker-compose.yml my-app

# Scale a service across the cluster
docker service scale my-app_api=5

# View services and their status
docker service ls
docker service ps my-app_api

```

## Kubernetes (K8s)

Kubernetes is the **industry standard** for container orchestration. It's more complex but far more powerful and extensible.

## Key Kubernetes Concepts

| Concept                   | What it is                                                                                                    | Docker equivalent                      |
|---------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------|
| <b>Pod</b>                | Smallest deployable unit. One or more containers that share network/storage.                                  | A container (roughly)                  |
| <b>Deployment</b>         | Manages a set of identical Pods. Handles scaling and rolling updates.                                         | <code>docker service</code>            |
| <b>Service</b>            | Stable network endpoint that routes traffic to Pods.                                                          | Port mapping + load balancer           |
| <b>Namespace</b>          | Virtual cluster for isolating resources (e.g., <code>dev</code> , <code>staging</code> , <code>prod</code> ). | No direct equivalent                   |
| <b>ConfigMap / Secret</b> | External configuration and sensitive data, injected into Pods.                                                | Environment variables / Docker secrets |
| <b>Ingress</b>            | Routes external HTTP traffic to internal Services based on rules.                                             | Reverse proxy (nginx)                  |

## When to Use Which?

| Criteria   | Docker Swarm                   | Kubernetes           |
|------------|--------------------------------|----------------------|
| Complexity | Simple, minimal learning curve | Steep learning curve |
| Setup time | Minutes                        | Hours to days        |



| Criteria        | Docker Swarm                       | Kubernetes                                      |
|-----------------|------------------------------------|-------------------------------------------------|
| Scale           | Small to medium (up to ~100 nodes) | Any scale (thousands of nodes)                  |
| Ecosystem       | Limited                            | Massive (Helm, operators, service meshes, etc.) |
| Best for        | Small teams, simple architectures  | Large teams, complex microservices              |
| Managed options | Limited                            | EKS, GKE, AKS (all major clouds)                |



**Practical advice:** Start with Docker Swarm if you have a small team and fewer than 10 services. Move to Kubernetes when you need advanced features like auto-scaling, custom resource definitions, or a rich ecosystem of tools.

## 12.4 Service Mesh

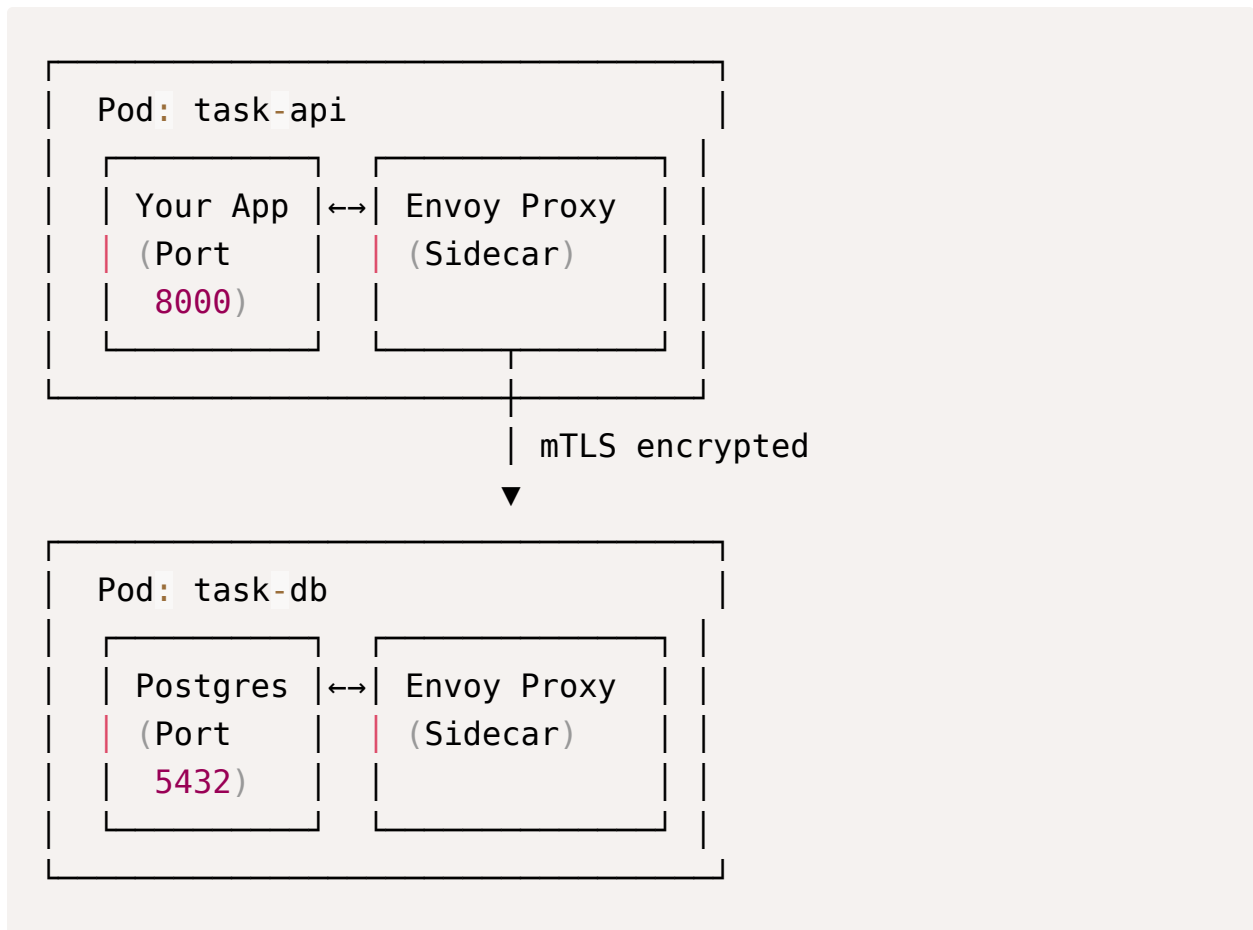
As your microservices grow, managing **service-to-service communication** becomes a challenge. A **service mesh** handles this for you transparently.

### What Problems Does a Service Mesh Solve?

1. **Traffic management** — Route requests between service versions (canary deployments, A/B testing)
2. **Security** — Automatic mutual TLS (mTLS) between services. No more managing certificates manually.
3. **Observability** — Distributed tracing, metrics, and logging for every request between services
4. **Reliability** — Automatic retries, timeouts, circuit breakers

### How It Works

A service mesh injects a **sidecar proxy** (usually Envoy) alongside each service container. All traffic flows through this proxy, which enforces policies without changing your application code.



## Popular Service Meshes

| Mesh                  | Strengths                                                   | Best for                                                   |
|-----------------------|-------------------------------------------------------------|------------------------------------------------------------|
| <b>Istio</b>          | Feature-rich, strong traffic management, large community    | Large-scale production, teams needing full control         |
| <b>Linkerd</b>        | Lightweight, simple, low resource overhead                  | Teams wanting simplicity without sacrificing core features |
| <b>Consul Connect</b> | Integrates with HashiCorp ecosystem, multi-platform support | Hybrid environments (VMs + containers)                     |



**Do you need a service mesh?** Not always. If you have fewer than 5-10 services, the complexity overhead isn't worth it. Start with a service mesh when you need mTLS everywhere, fine-grained traffic control, or deep observability across dozens of services.

## 12.5 Monitoring & Observability

You can't fix what you can't see. Monitoring tells you *what's happening*.

Observability tells you *why it's happening*.

### The Three Pillars

1. **Metrics** — Numerical measurements over time (CPU usage, request count, error rate)
2. **Logs** — Detailed event records from your application
3. **Traces** — The journey of a request across multiple services

### Prometheus + Grafana (Metrics)

**Prometheus** scrapes metrics from your services. **Grafana** visualizes them in dashboards.

```
# docker-compose.monitoring.yml
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    ports:
      - "9090:9090"
    networks:
      - monitoring

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
```

```

    - grafana-data:/var/lib/grafana
  networks:
    - monitoring

networks:
  monitoring:
    driver: bridge

volumes:
  grafana-data:

```

```

# prometheus.yml – tell Prometheus what to scrape
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "task-api"
    static_configs:
      - targets: ["task-api:8000"]

  - job_name: "node-exporter"
    static_configs:
      - targets: ["node-exporter:9100"]

```

## ELK Stack (Logs)

The **ELK stack** (Elasticsearch + Logstash + Kibana) is the standard for centralized logging.



A lighter alternative is **Loki + Grafana**, Loki indexes only log metadata (not full text), making it cheaper and faster for most use cases.

## Jaeger (Distributed Tracing)

When a user request touches 5 different microservices, tracing shows you exactly where time is spent and where failures occur.

```
# Run Jaeger locally
docker run -d --name jaeger \
  -p 16686:16686 \
  -p 4317:4317 \
  jaegertracing/all-in-one:latest
```

```
# Open the Jaeger UI
# http://localhost:16686
```



**Starter stack recommendation:** Prometheus + Grafana for metrics, Loki for logs, and Jaeger for traces. All can run in Docker containers and integrate with each other through Grafana dashboards.

## 12.6 Infrastructure as Code (IaC)

Manually clicking through the AWS console doesn't scale. **Infrastructure as Code** lets you define your entire cloud infrastructure in version-controlled files.

### Terraform

**Terraform** by HashiCorp is the most popular multi-cloud IaC tool. You describe *what* you want, and Terraform figures out *how* to create it.

```
# main.tf – Define your ECS infrastructure
```

```

provider "aws" {
  region = "us-east-1"
}

# Create a VPC
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "task-manager-vpc"
  }
}

# Create an ECS cluster
resource "aws_ecs_cluster" "main" {
  name = "task-manager-cluster"
}

# Create an ECR repository
resource "aws_ecr_repository" "app" {
  name = "task-manager"
  image_tag_mutability = "Mutable"

  image_scanning_configuration {
    scan_on_push = true
  }
}

# Create an ECS service with Fargate
resource "aws_ecs_service" "api" {
  name = "task-api-service"
  cluster = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.api.arn
  desired_count = 2
  launch_type = "FARGATE"

  network_configuration {

```

```

    subnets          = aws_subnet.public[*].id
    security_groups    = [aws_security_group.api.id]
    assign_public_ip   = true
  }
}

```

## Terraform Workflow

```

# Initialize Terraform (download provider plugins)
terraform init

# Preview what Terraform will create/change
terraform plan

# Apply the changes (create/update infrastructure)
terraform apply

# Destroy everything (tear down infrastructure)
terraform destroy

```

## AWS CloudFormation

CloudFormation is AWS's **native** IaC tool. It uses YAML or JSON templates.

```

# cloudformation-template.yml
AWSTemplateFormatVersion: "2010-09-09"
Description: Task Manager ECS Stack

Resources:
  ECSCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: task-manager-cluster

  TaskDefinition:

```

```

Type: AWS::ECS::TaskDefinition
Properties:
  Family: task-manager
  Cpu: "256"
  Memory: "512"
  NetworkMode: awsvpc
  RequiresCompatibilities:
    - FARGATE
  ContainerDefinitions:
    - Name: task-api
      Image: !Sub "${AWS::AccountId}.dkr.ecr.us-east-1.amazonaws.com/task-manager:latest"
      PortMappings:
        - ContainerPort: 8000

```

```

# Deploy with CloudFormation
aws cloudformation create-stack \
  --stack-name task-manager \
  --template-body file://cloudformation-template.yml

# Update an existing stack
aws cloudformation update-stack \
  --stack-name task-manager \
  --template-body file://cloudformation-template.yml

# Delete the stack
aws cloudformation delete-stack --stack-name task-manager

```

## Terraform vs CloudFormation

| Criteria      | Terraform                              | CloudFormation |
|---------------|----------------------------------------|----------------|
| Cloud support | Multi-cloud (AWS, GCP, Azure, etc.)    | AWS only       |
| Language      | HCL (HashiCorp Configuration Language) | YAML or JSON   |



| Criteria         | Terraform                                 | CloudFormation                           |
|------------------|-------------------------------------------|------------------------------------------|
|                  | Language)                                 |                                          |
| State management | Requires a state file (local or remote)   | Managed by AWS automatically             |
| Community        | Large, thousands of providers and modules | AWS-focused                              |
| Best for         | Multi-cloud or cloud-agnostic teams       | AWS-only teams wanting tight integration |



**Golden rule of IaC:** If you created it by clicking in a console, it will eventually break and no one will know how to recreate it. If you defined it in code, it's versioned, reviewable, and reproducible. **Always use IaC for production infrastructure.**

That covers the complete journey ,from understanding what a container is, to building and deploying production applications, to scaling and monitoring them in the cloud. **The best way to learn is to build.** Pick a project, containerize it, deploy it, and break things along the way. That's how the knowledge sticks.

Over to you.